Cahier des charges – Application ATS (Gestion RH & Recrutement)

Version 1.0 – Novembre 2026 – Auteur : Ahmed

1. Présentation du projet

1.1 Contexte

Les services RH gèrent souvent les offres, les candidatures et les entretiens avec des outils dispersés (emails, tableurs, messageries). Ce projet vise à centraliser tout le processus de recrutement dans une application web unique : un **ATS (Applicant Tracking System)**.

1.2 Objectifs

- Centraliser la publication des offres et la réception des candidatures.
- Suivre chaque candidat dans un pipeline visuel (kanban).
- Planifier les entretiens et collecter les feedbacks des managers.
- Notifier automatiquement les parties prenantes.
- Fournir des indicateurs de pilotage au service RH.
- Démontrer une architecture professionnelle (conteneurisée, déployable sur le cloud, testée, documentée) dans un contexte portfolio.

1.3 Périmètre

Inclus : gestion des utilisateurs et rôles, offres d'emploi, candidatures, upload et consultation de CV, pipeline kanban, entretiens, notifications email, dashboard, déploiement cloud.

Hors périmètre (v1) : parsing automatique de CV par IA, paiement, application mobile native, signature électronique, multi-entreprises (multi-tenant).

2. Acteurs et rôles

Rôle	Description	Droits principaux
Candidat	Personne qui postule	Consulter les offres, postuler, suivre ses candidatures, gérer son profil

Rôle	Description	Droits principaux
RH	Recruteur	CRUD offres, voir tous les candidats, gérer le pipeline, planifier des entretiens, dashboard
Manager	Responsable d'équipe	Voir les candidatures de ses offres, donner un feedback, participer aux entretiens
Admin	Administrateur technique	Gérer les utilisateurs et les rôles, paramètres généraux, audit

3. Besoins fonctionnels

3.1 Authentification et comptes

- RF-01 : inscription candidat par email + mot de passe, avec vérification d'email.
- RF-02 : connexion avec JWT (access token + refresh token).
- RF-03 : réinitialisation du mot de passe par email.
- RF-04 : gestion du profil (nom, téléphone, photo, LinkedIn).
- RF-05 : l'Admin peut créer des comptes RH/Manager, activer/désactiver un compte et changer un rôle.

3.2 Offres d'emploi

- RF-10 : créer, modifier, supprimer une offre (titre, description, département, lieu, type de contrat, fourchette de salaire optionnelle, compétences requises, date limite).
- RF-11 : statuts d'une offre : Brouillon, Publiée, Clôturée, Archivée.
- RF-12 : liste publique des offres avec pagination, recherche par mot-clé et filtres (lieu, contrat, département).
- RF-13 : assigner un manager responsable à une offre.

3.3 Candidatures

- RF-20 : le candidat postule à une offre avec CV (PDF, 5 Mo max) et lettre de motivation optionnelle.
- RF-21 : un candidat ne peut postuler qu'une fois par offre.
- RF-22 : le candidat suit le statut de ses candidatures.
- RF-23 : le candidat peut retirer sa candidature.
- RF-24 : la RH consulte, télécharge le CV, ajoute des notes internes et des tags.
- RF-25 : recherche et filtres sur les candidatures (offre, statut, date, tag).

3.4 Pipeline de recrutement (kanban)

- RF-30 : étapes par défaut : Reçue → Présélection → Entretien RH → Entretien technique → Offre → Embauché / Refusé.
- RF-31 : déplacer une candidature d'une étape à l'autre par drag & drop.
- RF-32 : chaque changement d'étape est historisé (qui, quand, ancienne et nouvelle étape).
- RF-33 : un motif est obligatoire en cas de refus.

3.5 Entretiens

- RF-40 : planifier un entretien (date, heure, durée, type visio/présentiel, lien ou lieu, participants).
- RF-41 : le candidat et les participants reçoivent une invitation par email.
- RF-42 : le manager saisit un feedback (note de 1 à 5, points forts, points faibles, recommandation).
- RF-43 : vue calendrier des entretiens pour la RH et les managers.

3.6 Notifications

- RF-50 : emails automatiques : confirmation de candidature, changement de statut, invitation à un entretien, rappel 24 h avant.
- RF-51 : notifications in-app (cloche) pour la RH et les managers.
- RF-52 : modèles d'emails paramétrables.

3.7 Dashboard et reporting

- RF-60 : nombre de candidatures par offre et par période.
- RF-61 : taux de conversion par étape du pipeline.
- RF-62 : temps moyen de recrutement (time-to-hire).
- RF-63 : répartition par source de candidature.
- RF-64 : export CSV des candidatures.

3.8 Administration et audit

- RF-70 : journal d'audit des actions sensibles (création/suppression, changement de rôle, refus, export).
- RF-71 : gestion des utilisateurs (liste, recherche, activation).

4. Besoins non fonctionnels

Catégorie	Exigence
Performance	Temps de réponse API < 500 ms pour 95 % des requêtes courantes ; pagination systématique
Sécurité	Mots de passe hachés (BCrypt), JWT signé, RBAC, protection CSRF/XSS/injection, validation des entrées, HTTPS en production
Confidentialité	Conformité RGPD : consentement, droit d'accès et de suppression des données candidat, durée de conservation configurable
Disponibilité	Cible 99 % en production, healthchecks et redémarrage automatique des conteneurs
Scalabilité	Backend stateless, déployable en plusieurs instances derrière un load balancer
Maintenabilité	Architecture en couches, couverture de tests ⩾ 70 % sur la couche service, code documenté
Portabilité	Environnement identique en local et en production via Docker
Ergonomie	Interface responsive (desktop, tablette, mobile), accessibilité de base (contraste, navigation clavier)
Observabilité	Logs structurés, métriques Prometheus, dashboards Grafana

5. Contraintes et choix techniques

5.1 Stack

Couche	Technologie
Backend	Java 21, Spring Boot 3/4, Spring Security (JWT), Spring Data JPA, Bean Validation, MapStruct, Flyway

Couche	Technologie
Frontend	Angular (standalone components, lazy loading), Angular Material ou PrimeNG, Angular CDK (drag & drop)
Base de données	PostgreSQL 16
Cache	Redis
Stockage fichiers	MinIO en local, AWS S3 ou Google Cloud Storage en production
Emails	MailHog en local, AWS SES ou SMTP en production
Documentation API	SpringDoc OpenAPI (Swagger UI)
Tests	JUnit 5, Mockito, Testcontainers, Jasmine/Karma ou Jest, Cypress ou Playwright (E2E)

5.2 Architecture

- Application **monolithique modulaire** en v1 (un module par domaine métier : auth, job, application, interview, notification, storage), avec possibilité d'extraire le module notification en microservice par la suite.
- API REST versionnée (`/api/v1/...`), échanges JSON.
- Pattern `StorageService` (interface) avec implémentations MinIO/S3 et GCS, sélectionnées par profil Spring.
- Frontend SPA communiquant avec l'API via un interceptor JWT.

5.3 Infrastructure et DevOps

- **Docker** : Dockerfile multi-stage pour le backend (build Maven puis JRE léger) et le frontend (build Node puis Nginx).
- **Docker Compose** : services `postgres` , `redis` , `minio` , `mailhog` , `backend` , `frontend` , `pgadmin` (profil dev).
- **CI/CD** : GitHub Actions (build, tests, analyse qualité, build des images, push vers ECR ou Artifact Registry, déploiement).
- **Cloud** :
  - Option AWS : ECS (Fargate) ou EC2, RDS PostgreSQL, S3, SES, Secrets Manager, CloudWatch.
  - Option GCP : Cloud Run, Cloud SQL, Cloud Storage, Secret Manager, Cloud Logging.

- **Secrets** : jamais dans le dépôt ; variables d'environnement et gestionnaire de secrets du cloud.

6. Modèle de données (entités principales)

Entité	Attributs clés	Relations
User	id, email, passwordHash, firstName, lastName, phone, role, enabled, createdAt	1 User → N Applications (candidat)
JobOffer	id, title, description, department, location, contractType, salaryMin/Max, status, deadline, createdAt	N-1 User (créateur RH), N-1 User (manager), 1-N Applications
Application	id, coverLetter, cvFileKey, currentStage, source, createdAt	N-1 JobOffer, N-1 User (candidat), 1-N StageHistory, 1-N Note, 1-N Interview
StageHistory	id, fromStage, toStage, reason, changedAt	N-1 Application, N-1 User
Note	id, content, createdAt	N-1 Application, N-1 User
Interview	id, startAt, durationMin, mode, locationOrLink, status	N-1 Application, N-N User (participants), 1-1 Feedback
Feedback	id, rating, strengths, weaknesses, recommendation	1-1 Interview, N-1 User
Notification	id, type, message, read, createdAt	N-1 User
AuditLog	id, action, entityType, entityId, actor, createdAt	N-1 User

7. API REST (aperçu)

Méthode	Endpoint	Description	Rôles
POST	/api/v1/auth/register	Inscription candidat	Public
POST	/api/v1/auth/login	Connexion	Public
POST	/api/v1/auth/refresh	Renouvellement du token	Public

Méthode	Endpoint	Description	Rôles
GET	/api/v1/jobs	Liste des offres publiées	Public
POST	/api/v1/jobs	Créer une offre	RH
PUT	/api/v1/jobs/{id}	Modifier une offre	RH
POST	/api/v1/jobs/{id}/applications	Postuler	Candidat
GET	/api/v1/applications	Lister les candidatures	RH, Manager
PATCH	/api/v1/applications/{id}/stage	Changer d'étape	RH
POST	/api/v1/applications/{id}/notes	Ajouter une note	RH, Manager
POST	/api/v1/interviews	Planifier un entretien	RH
POST	/api/v1/interviews/{id}/feedback	Donner un feedback	Manager
GET	/api/v1/dashboard/stats	Statistiques	RH
GET	/api/v1/admin/audit	Journal d'audit	Admin

8. Écrans principaux (frontend)

- **Public** : accueil, liste des offres, détail d'une offre, connexion/inscription.
- **Candidat** : mes candidatures, profil.
- **RH** : gestion des offres, liste des candidatures, fiche candidat, pipeline kanban, calendrier des entretiens, dashboard.
- **Manager** : mes offres, candidatures à évaluer, feedbacks.
- **Admin** : utilisateurs, audit, paramètres.

9. Sécurité et conformité

- Autorisation par rôle côté backend (`@PreAuthorize`) et côté frontend (guards).
- Un candidat ne peut accéder qu'à ses propres données.
- Validation du type et de la taille des fichiers uploadés (PDF uniquement), analyse du nom, stockage hors du système de fichiers applicatif.
- URLs signées à durée limitée pour le téléchargement des CV.
- Limitation du débit (rate limiting) sur les endpoints d'authentification.
- Suppression ou anonymisation des données d'un candidat sur demande (RGPD).
- Dépendances scannées (Dependabot ou équivalent).

10. Tests et qualité

- Tests unitaires (services) avec JUnit 5 et Mockito.
- Tests d'intégration (repositories, contrôleurs) avec Testcontainers PostgreSQL.
- Tests frontend : composants et services, puis scénarios E2E sur les parcours clés (postuler, déplacer une candidature, planifier un entretien).
- Analyse statique (SonarQube/SonarCloud ou équivalent) dans la CI.
- Conventions : Conventional Commits, branches `main` / `develop` / `feature/*`, revues via pull requests.

11. Planning prévisionnel

Phase	Contenu	Durée estimée
0	Cahier des charges, maquettes, modèle de données	1 semaine
1	Socle : Docker Compose, squelette back/front, authentification JWT et rôles	1 à 2 semaines
2	Offres et candidatures, upload de CV	2 semaines
3	Pipeline kanban, historique, notes, entretiens et feedbacks	2 semaines
4	Notifications email et in-app, dashboard	1 à 2 semaines
5	Tests, CI/CD, déploiement cloud, monitoring	1 à 2 semaines
6	Documentation, README, démo, captures	1 semaine

12. Livrables

- Code source (dépôt Git) : backend, frontend, `docker-compose.yml`, Dockerfiles.
- Documentation API (Swagger) et README d'installation.
- Diagrammes : cas d'utilisation, modèle de données (ERD), architecture de déploiement.
- Pipeline CI/CD fonctionnel.
- Application déployée sur le cloud avec une URL de démonstration.
- Jeu de données de démonstration (seed).

13. Évolutions possibles (v2)

- Parsing de CV et scoring automatique par IA.
- Intégration de calendriers (Google Calendar, Outlook).
- Multi-entreprises (multi-tenant).
- Page carrière personnalisable et diffusion vers des job boards.
- Notifications temps réel via WebSocket.
- Extraction du module de notification en microservice avec message broker (Kafka ou RabbitMQ).